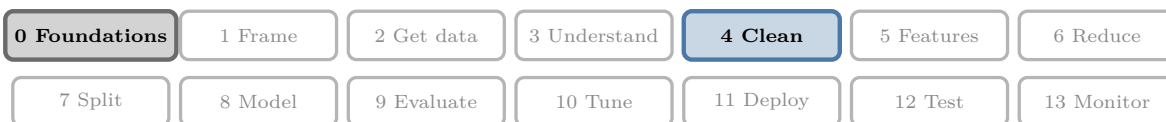


Outlier Detection with the Z-score Method

Contents

1. Overview	2
2. The condition: a normal feature	3
3. The 68-95-99.7 rule	3
3.1 The limits	4
4. Why it is called the z-score method	4
5. Treating the outliers: trimming or capping	5
6. The placement data	6
7. Finding the limits and the outliers	7
8. Trimming in code	8
9. Trimming with the z-score column	9
10. Capping in code	9
11. Learning the limits on the training set	11
12. Strengths and limits of the z-score method	12
13. Summary	14
14. Sources	14
15. Key terms	14

Where this fits: Pipeline map steps 0 (Foundations), 4 (Clean). See the [Course map](#).



- **Builds on:** Descriptive statistics ([Note 19](#)); Skewness ([Note 20](#)); Probability density function (PDF) ([Note 20](#)); Standardization ([Note 24](#)); Outliers ([Note 41](#)); Percentile outlier method ([Note 41](#)).
- **Leads to:** Naive Bayes ([Note 87](#)); Standard normal and the z-table ([Note 251](#)); Q-Q plot ([Note 260](#)); Central limit theorem ([Note 271](#)); Likelihood ([Note 630](#)); Multivariate normal distribution ([Note 640](#)).
- **Compare with:** IQR outlier method ([Note 41](#)); Percentile outlier method ([Note 41](#)); Student's t-distribution ([Note 282](#)).

1. Overview

Key point: For a roughly normal feature, every value more than 3 standard deviations from the mean is an outlier; we then trim those rows or cap the values at the limits.

Note 41 listed three rules for detecting outliers. This Note puts the first one to work: the mean ± 3 standard deviations rule, also called the **z-score method** (G-2139). The z-score method only fits a **feature** (G-772) (an input variable, one column of the data table) that is normal or close to normal. Each **observation** (G-1374) is one record (one row), here one student.

Figure 1 shows the whole method, in three steps:

1. check the shape of the column;
2. compute a lower and an upper limit;
3. treat the values outside them by trimming or capping (both defined in Note 41, Section 7).

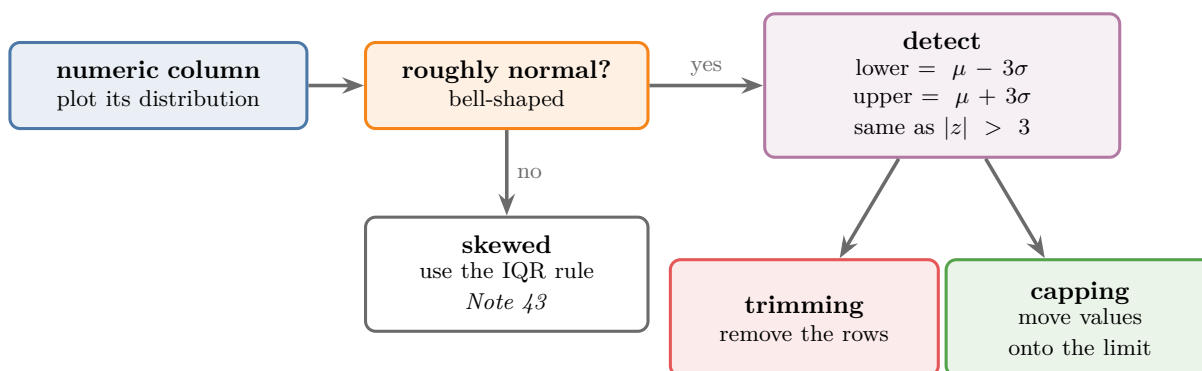


Figure 1: The z-score method: check the column is normal, compute the limits, then trim or cap

2. The condition: a normal feature

Key point: The z-score method works only on a column that is normally distributed, or almost.

A **normal distribution** (G-1343) has the shape of a bell, so its curve is also called the **bell curve** (G-274). Most values sit near the centre, and fewer and fewer values lie towards the two ends.

People's heights and marks in an exam often follow this shape, at least roughly.

So before we use the method, we plot the feature and check that it looks like a bell. A strongly skewed feature needs another rule, the IQR fences of Note 43.

3. The 68-95-99.7 rule

Key point: In a normal feature, about 68.3% of the values lie within 1 standard deviation of the mean, 95.4% within 2, and 99.7% within 3.

A normal distribution always spreads its values in the same fixed way around its mean. Write μ for the mean and σ for the standard deviation. Figure 2 shows the three ranges:

- from $\mu - \sigma$ to $\mu + \sigma$: about 68.3% of the values;
- from $\mu - 2\sigma$ to $\mu + 2\sigma$: about 95.4%;
- from $\mu - 3\sigma$ to $\mu + 3\sigma$: about 99.7%.

This spread is called the **68-95-99.7 rule** (or the **empirical rule** (G-53)). The rule holds for every normal feature, whatever its mean and standard deviation.

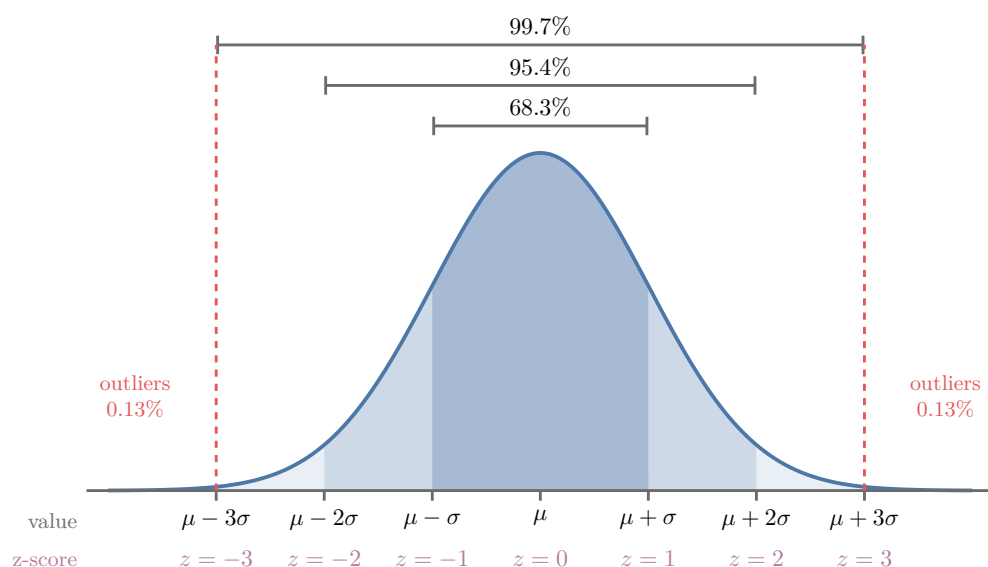


Figure 2: The 68-95-99.7 rule; the red tails beyond 3 standard deviations hold only 0.3% of the values

Only about 0.3% of the values lie beyond 3 standard deviations, about 0.13% on each side. A value out there is so rare that the usual practice is to call it an outlier: like a 2.3-metre-tall person in a crowd, possible but so unusual that we look twice.

Extra: The exact numbers are 68.27%, 95.45% and 99.73%. They come from the area under the bell curve, and they are only true for a perfectly normal feature. A real column that is close to normal gives numbers close to them: the **cgpa** column of Section 6 has 67.9%, 95.7% and 99.5% of its values in the three ranges (last cell of the Notebook).

3.1 The limits

Key point: The lower limit is $\mu - 3\sigma$ and the upper limit is $\mu + 3\sigma$; values outside them are outliers.

The rule turns into two limits, step by step:

1. **In words:** step three standard deviations below the mean for the lower limit, and three above for the upper limit.
2. **Formula:**

$$\text{lower} = \mu - 3\sigma, \quad \text{upper} = \mu + 3\sigma$$

3. **Example:** the **cgpa** column of Section 6 has mean 6.96 and standard deviation 0.62:

$$\text{lower} = 6.96 - 3 \times 0.62 = 5.11, \quad \text{upper} = 6.96 + 3 \times 0.62 = 8.81$$

A CGPA below 5.11 or above 8.81 is an outlier. (With the unrounded numbers, the limits are 5.1135 and 8.8089.)

4. Why it is called the z-score method

Key point: A value's z-score counts how many standard deviations it lies from the mean, so “outside mean ± 3 standard deviations” is the same as “z-score above 3 or below -3”.

The **z-score** (G-2141) of a value is the standardization formula of Note 24 (Section 5), applied to one value:

1. **In words:** subtract the mean of the column, then divide by its standard deviation.
2. **Formula:**

$$z = \frac{x - \mu}{\sigma}$$

3. **Example:** for a CGPA of 9.12 (mean 6.9612, standard deviation 0.6159),

$$z = \frac{9.12 - 6.9612}{0.6159} = 3.51$$

and for a CGPA of 4.89,

$$z = \frac{4.89 - 6.9612}{0.6159} = -3.36.$$

Converting the whole column to z-scores gives the bottom row of numbers in Figure 2. The limit $\mu + 3\sigma$ becomes $z = 3$, and $\mu - 3\sigma$ becomes $z = -3$. Figure 3 does the conversion on the real `cgpa` column; watch the two dashed limits travel with the data and land exactly on -3 and $+3$, with the same five red students still outside.

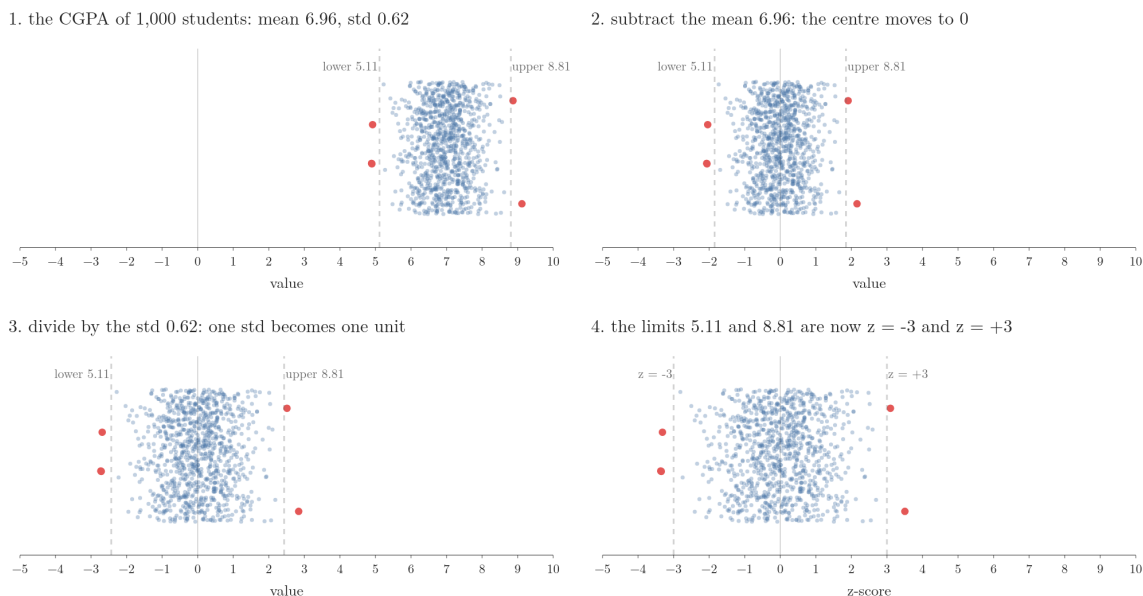


Figure 3: Subtracting the mean slides the CGPA column to 0; dividing by the standard deviation squeezes it, so the limits 5.11 and 8.81 become $z = -3$ and $z = +3$

So there are two equal ways to detect outliers:

- compute the limits $\mu \pm 3\sigma$ and compare each value with them;
- compute each value's z-score and flag every value with $z > 3$ or $z < -3$.

Both flag exactly the same rows. Section 9 does it the second way.

5. Treating the outliers: trimming or capping

Key point: Trimming deletes the outlier rows; capping replaces each outlier with the limit it crossed.

Both treatments, **trimming** (G-2019) and **capping** (G-345), come from the [outliers Note](#) (section seven, ways to treat outliers). With the limits of Section 3.1, capping turns a CGPA of 9.12 into 8.81 and a CGPA of 4.89 into 5.11.

6. The placement data

Key point: The data has 1,000 students; their CGPA is close to normal, but their placement exam marks are skewed, so the method is used on CGPA only.

The data comes from a college: one observation per student, 1,000 students, three columns.

cgpa	placement_exam_marks	placed
7.19	26	1
7.46	38	1
6.42	8	1
7.23	17	0

- **cgpa**: the student's CGPA (out of 10) before the placement season.
- **placement_exam_marks**: marks out of 100 in the aptitude test that companies hold before placement (aptitude, coding, English).
- **placed**: 1 if the student got a job offer, 0 if not. This column is the **target** (G-1949), the output a model would predict.

Two features are candidates for outlier detection: **cgpa** and **placement_exam_marks**. Figure 4 plots the distribution of each.

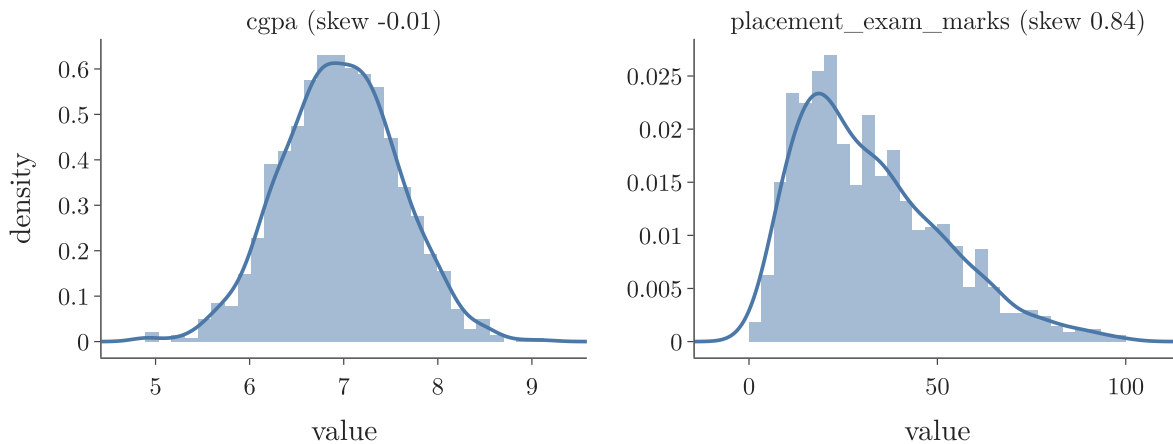


Figure 4: CGPA is bell-shaped; the placement exam marks have a long tail to the right

- **cgpa** is close to normal: a bell, with skewness -0.01 (0 means perfectly symmetric, Note 20).
- **placement_exam_marks** is right-skewed (skewness 0.84): many students scored low and only a few scored high.

So the z-score method can only be used on **cgpa**. The marks feature needs the IQR rule of Note 43.

Python: Loading the data and checking the shape of the features.

```
import pandas as pd
```

```
df = pd.read_csv("data/placement.csv")  
df.shape      # (1000, 3)
```

```
# skewness: near 0 = symmetric  
df[["cgpa", "placement_exam_marks"]].skew()  
# cgpa -0.01, placement_exam_marks 0.84
```

The Notebook draws Figure 4 with Seaborn's objects interface: `so.Hist` for the bars and `so.KDE` for the smooth curve.

7. Finding the limits and the outliers

Key point: CGPA has mean 6.96 and standard deviation 0.62, which gives limits of 5.11 and 8.81 and flags 5 students.

First we look at four numbers of the `cgpa` column:

Mean	Standard deviation	Minimum	Maximum
6.96	0.62	4.89	9.12

The limits are $\text{mean} \pm 3 \text{ standard deviations}$, as in Section 3.1: **5.11** and **8.81**. The minimum, 4.89, is below the lower limit and the maximum, 9.12, is above the upper limit, so the column does have outliers.

Five students fall outside the limits (Figure 5, red):

Row	cgpa	placement_exam_marks	placed
485	4.92	44	1
995	8.87	44	1
996	9.12	65	1
997	4.89	34	0
999	4.90	10	1

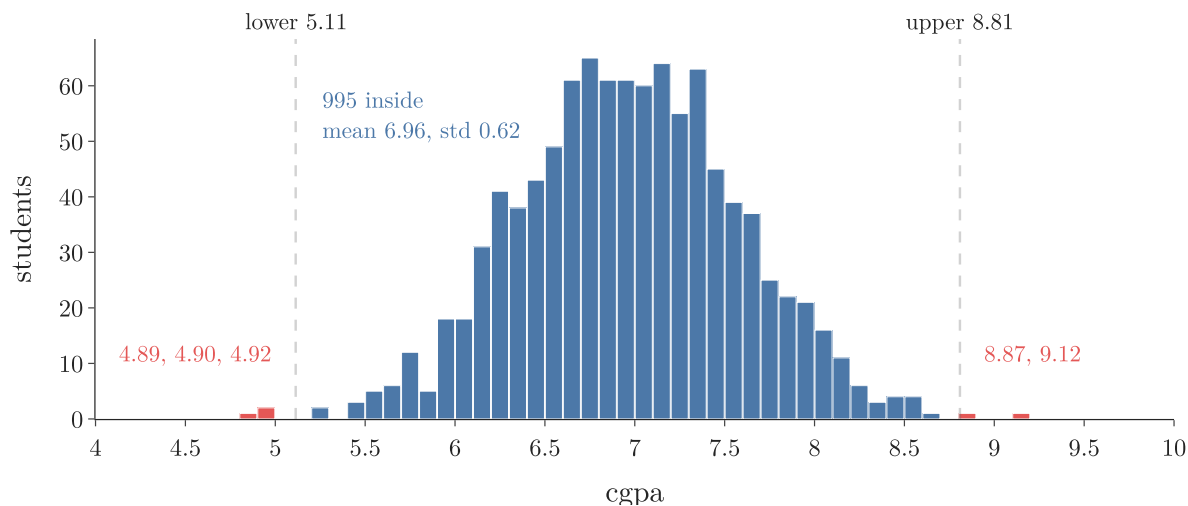


Figure 5: The CGPA of 1,000 students with the limits 5.11 and 8.81; the 5 red values lie outside

Four of the five were placed, including two of the three students with a CGPA below 5.

Python: Computing the limits and selecting the outliers.

```
mean = df["cgpa"].mean()      # 6.9612
std = df["cgpa"].std()        # 0.6159

upper_limit = mean + 3 * std   # 8.8089
lower_limit = mean - 3 * std   # 5.1135

# | means "or": too high or too low
is_outlier = ((df["cgpa"] > upper_limit)
              | (df["cgpa"] < lower_limit))
df[is_outlier]                # the 5 rows above
```

8. Trimming in code

Key point: Keeping only the rows inside the limits removes the 5 outliers and leaves 995 rows.

Trimming is a filter: keep only the rows whose CGPA lies between the two limits. Figure 6 (middle) shows the result: the same bell, without the five values at the ends.

Python: Trimming.

```
# ~ means "not": keep the rows that are not outliers
trimmed = df[~is_outlier]
trimmed.shape    # (995, 3)
```

Extra: It is safer to use the variables `upper_limit` and `lower_limit` than to type rounded limits such as 8.80 and 5.11 by hand. A rounded upper limit of 8.80 is slightly below the true 8.8089, so a CGPA of 8.805 would wrongly count as an outlier. In this data no CGPA lies in that gap, so both ways give the same 995 rows.

9. Trimming with the z-score column

Key point: We add a column of z-scores and keep the rows whose z-score lies between -3 and 3; the result is the same 995 rows.

The second approach follows Section 4. We compute the z-score of every student's CGPA and store it in a new column, `cgpa_zscore`.

cgpa	cgpa_zscore
7.19	0.37
7.46	0.81
6.42	-0.88

Then we look for z-scores beyond ± 3 :

- $z > 3$: two students, CGPA 8.87 ($z = 3.10$) and 9.12 ($z = 3.51$);
- $z < -3$: three students, CGPA 4.92 ($z = -3.31$), 4.89 ($z = -3.36$) and 4.90 ($z = -3.35$).

These are the same five students as in Section 7. Trimming them again leaves 995 rows.

Python: The z-score column and trimming with it.

```
df["cgpa_zscore"] = (df["cgpa"] - mean) / std

df[df["cgpa_zscore"] > 3]      # 2 rows
df[df["cgpa_zscore"] < -3]    # 3 rows

# keep -3 <= z <= 3
trimmed_z = df[df["cgpa_zscore"].between(-3, 3)]
trimmed_z.shape      # (995, 4)
```

10. Capping in code

Key point: `np.where` replaces each CGPA above 8.81 with 8.81 and each CGPA below 5.11 with 5.11; all 1,000 rows stay.

For capping we go through the column value by value:

- above the upper limit: replace it with the upper limit;
- below the lower limit: replace it with the lower limit;

- otherwise: leave it as it is.

NumPy's **np.where** does this for a whole column at once. **np.where** takes three things: a condition, the value to use where the condition is true, and the value to use where it is false. Two conditions need two **np.where** calls, one inside the other.

Python: Capping with **np.where**.

```
import numpy as np

# np.where(condition, if true, if false)
df["cgpa"] = np.where(
    df["cgpa"] > upper_limit,
    upper_limit,
    np.where(df["cgpa"] < lower_limit,
             lower_limit,
             df["cgpa"]))

df.shape          # (1000, 4): no row lost
df["cgpa"].describe() # min 5.1135, max 8.8089
```

The result, compared with the original column:

	Original	Capped
Rows	1,000	1,000
Mean	6.9612	6.9615
Standard deviation	0.6159	0.6127
Minimum	4.89	5.11
Maximum	9.12	8.81

The mean barely moves and the standard deviation shrinks a little. The minimum and maximum are now exactly the limits. Figure 6 (bottom) shows the three low values piled onto 5.11 and the two high values onto 8.81, in orange.

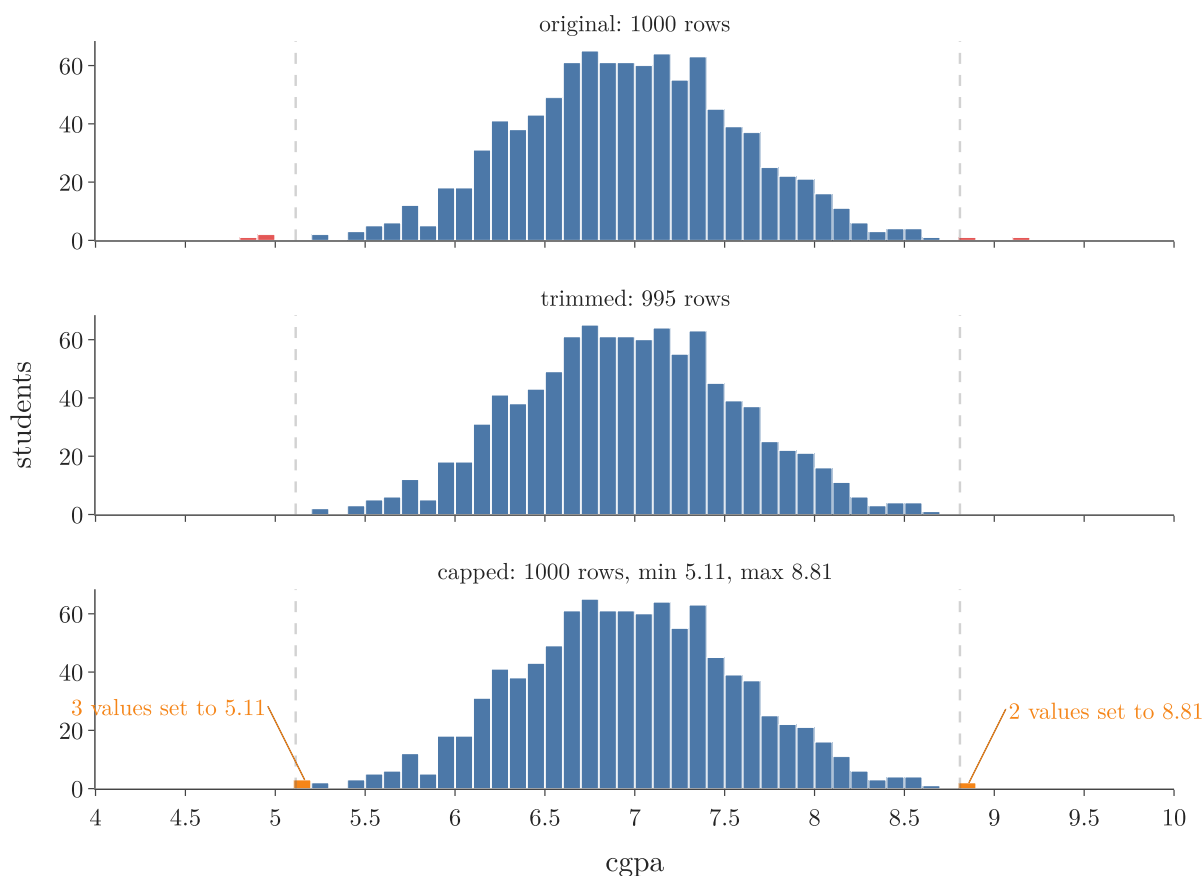


Figure 6: Before and after: the original column, the trimmed column (995 rows) and the capped column (1,000 rows, outliers moved onto the limits)

Extra: pandas has a one-line shortcut for capping: `df["cgpa"].clip(lower_limit, upper_limit)`. `clip` gives exactly the same column as the two nested `np.where` calls.

11. Learning the limits on the training set

Key point: The limits are learned from the data, so they should be learned on the training set only, like the mean and standard deviation of a scaler.

Extra: The steps above compute the mean and standard deviation on all 1,000 rows, before any train-test split. Computing them on all rows lets the test rows influence the limits, the same [data leakage](#) that the toy-project Note avoids for scaling (section seven). The cleaner order is:

1. Split the data into training and test sets.
2. Compute the mean, standard deviation and limits on the training set only.
3. Apply the same limits to the training and the test set.

With an 80/20 split (`random_state=42`), the 800 training rows have mean 6.95 and standard deviation 0.61. The training limits are **5.12** and **8.78**:

	Rows	Outliers (training limits)
Training set	800	4: CGPA 4.92, 4.89, 4.90, 9.12
Test set	200	1: CGPA 8.87

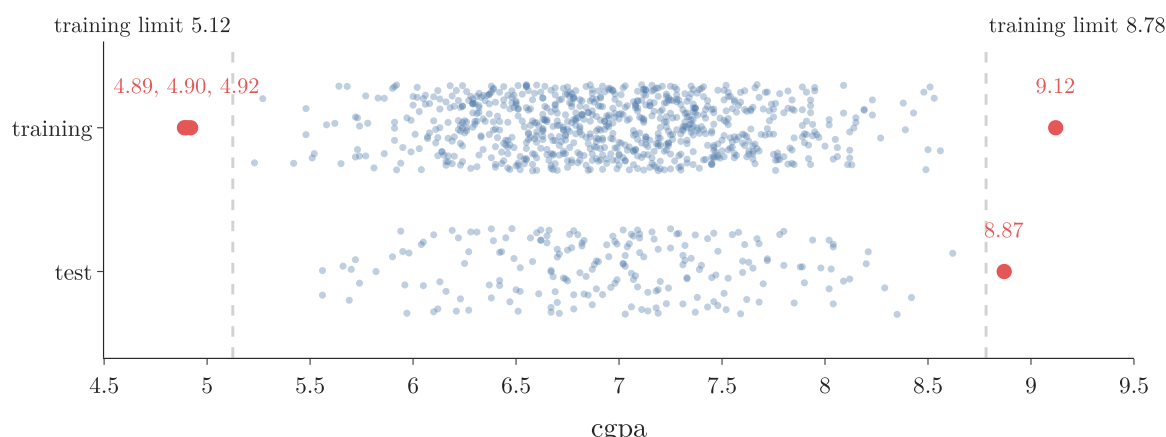


Figure 7: Limits learned on the 800 training rows (5.12 and 8.78), applied to both sets: 4 outliers in training, 1 in test

In Figure 7, watch the test student with CGPA 8.87: the training limits flag that student even though no test row helped set them.

The limits move only a little here, because 800 rows give almost the same mean and standard deviation as 1,000. Trimming is done on the training set only, since we cannot delete test rows (or real users) at prediction time. Capping can be applied to both sets with the training limits; the Notebook shows both steps.

12. Strengths and limits of the z-score method

Key point: The method is simple and effective, but only for a column that is roughly normal.

- **Simple:** two numbers (mean and standard deviation) give both limits.
- **Effective:** on a normal feature it flags exactly the rare values at the two ends.
- **Limited:** it assumes a normal feature. On a skewed feature the limits land in the wrong places. For the skewed marks feature they come out as -25.17 and 89.62 : the lower limit is below 0, so it can never flag anything, and 8 marks (0.8%, not 0.3%) are flagged on the high side. Note 43 handles this case with the IQR rule.

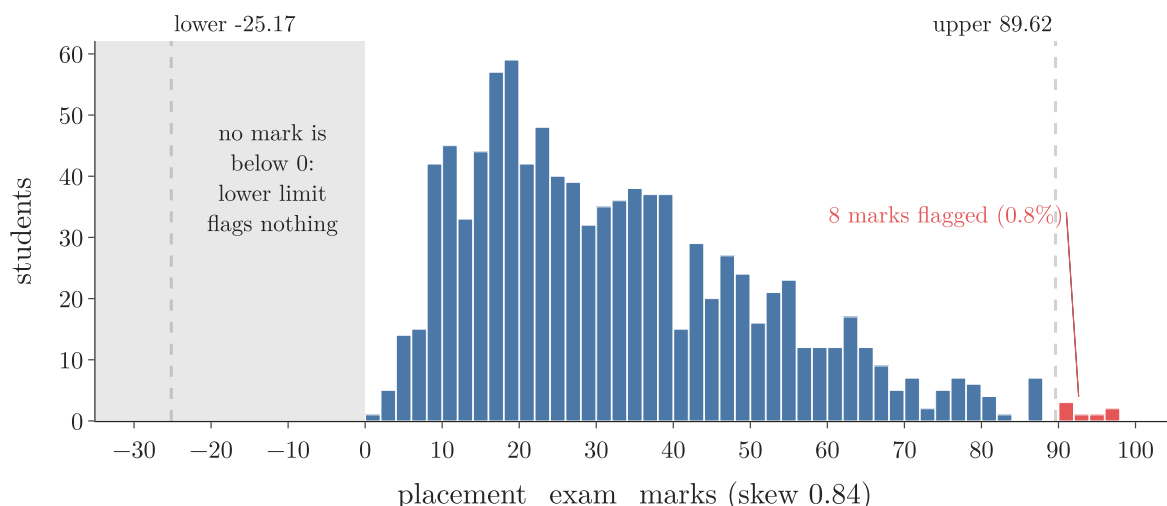


Figure 8: The same rule on the skewed marks: the lower limit -25.17 sits where no mark can be, and 8 high marks are flagged

In Figure 8, watch the grey band: half of the rule is spent on impossible negative marks, while the long right tail gets cut.

Extra: Two more things to keep in mind.

- **A perfect normal feature still has “outliers”.** About 0.27% of the values lie beyond 3 standard deviations by pure chance: about 3 in 1,000. In large data, the method always flags some real, valid values.
- **The outliers move the limits they are judged by.** The mean and standard deviation are computed from all the values, the extreme ones included, so a big outlier also widens its own limits. In a small sample this can hide it completely: with n values no z-score can be larger than $(n-1)/\sqrt{n}$ (Shiffler 1988; NIST 1.3.5.17). For 10 values that is 2.85, so the method can never flag anything, however extreme.

13. Summary

Step	What we do	On the placement data
Check the shape	plot the feature; it must be roughly normal	cgpa normal (skew -0.01); marks skewed (0.84), not used
Detect	limits $\mu \pm 3\sigma$, or $z > 3$ or $z < -3$	limits 5.11 and 8.81 ; 5 outliers
Trim	keep the rows inside the limits	995 rows left
Cap	move values beyond a limit onto it	1,000 rows; min 5.11 , max 8.81
Better practice	learn the limits on the training set	limits 5.12 and 8.78 ; 4 outliers in train, 1 in test

- The z-score method needs a feature that is normal or close to it.
- In a normal feature, about 68.3%, 95.4% and 99.7% of the values lie within 1, 2 and 3 standard deviations of the mean.
- Values beyond mean ± 3 standard deviations are outliers.
- The z-score $z = (x - \mu)/\sigma$ is the standardization formula; $|z| > 3$ flags exactly the same values.
- Trimming deletes the outlier rows; capping replaces each outlier with the limit, so no row is lost.
- The limits should be learned on the training set only.

14. Sources

Built from

- CampusX, “Outlier Detection and Removal using Z-score Method | Handling Outliers Part 2”, YouTube, <https://www.youtube.com/watch?v=OnPE-Z8jtqM>

Other references

- Shiffler, R. E. (1988). Maximum Z scores and outliers. *The American Statistician*, 42(1), 79–80.
- NIST/SEMATECH. *e-Handbook of Statistical Methods*, Section 1.3.5.17, “Detection of Outliers”. <https://www.itl.nist.gov/div898/handbook/>

15. Key terms

Term	Meaning
Feature	An input variable: one column of the data table

Term	Meaning
Observation	One record: one row of the data table
Target	The output a model predicts
Z-score method	Outlier detection that flags values more than 3 standard deviations from the mean; for roughly normal features
Normal distribution	A bell-shaped distribution: most values near the mean, fewer and fewer towards both ends
Bell curve	The curve of a normal distribution
68-95-99.7 rule (empirical rule)	In a normal feature, about 68.3%, 95.4% and 99.7% of values lie within 1, 2 and 3 standard deviations of the mean
Z-score	How many standard deviations a value lies from the mean: $(x - \mu)/\sigma$
Upper / lower limit	$\mu + 3\sigma$ and $\mu - 3\sigma$; values beyond them are outliers
<code>np.where</code>	NumPy function that picks one value where a condition is true and another where it is false